

Lecture 5: September 23

Demo -1 • MVP Planning • Team Charter • Mock Interviews

90 min lab

mentor time after

Agenda

Lab runs 6:10–7:40. After that, find a study room and meet your mentor remotely.

6:10–6:20

Demo -1 expectations

6:20–6:40

System architecture + technology plan

6:40–7:10

Team Charter workshop + AI usage agreement

7:10–7:35

Mock interview mini-round + job market strategy

7:35–7:40

Wrap-up and mentor meeting transition

Use the course website for current dates and assignment links.

Demo -1: what to prepare

Lightweight team progress check with instructors. No polished demo required yet.

1. Project proposal

Problem, users, value, competitors, and what makes your project technically meaningful.

2. Architecture diagram

Major components, data flow, external APIs, database, and key algorithms.

3. 10-week plan

MVP scope, milestones, risks, and who owns each subsystem.

**Teams should lead the meeting and be ready for feedback.
Show how you addressed major risks and open questions.
Working code is welcome, but not required for Demo -1.**

Example: Coffee Shop Crowding App

Use examples like this to make Demo -1 concrete.

Problem	Students/remote workers want to know how crowded a coffee shop is before going.
MVP users	Start with this SD class; expand to SEAS students, faculty, and staff after pilot.
Data plan	User-reported crowd levels first; investigate Google Popular Times only after access is verified.
Algorithm	Rank by estimated crowd level, distance, report freshness, seating/outlets, and user preference.
Risk	Low data volume, stale reports, unreliable external APIs, and overbuilding frontend.

From idea to MVP in 10 weeks

Build the smallest system that proves your project can work.

Do

- Build end-to-end early
- Validate data access
- Test with real users
- Keep UI simple
- Track risks every week

Do not

- Spend weeks on frontend polish
- Depend on unverified APIs
- Promise a huge product scope
- Wait until the end to integrate
- Submit code you cannot explain

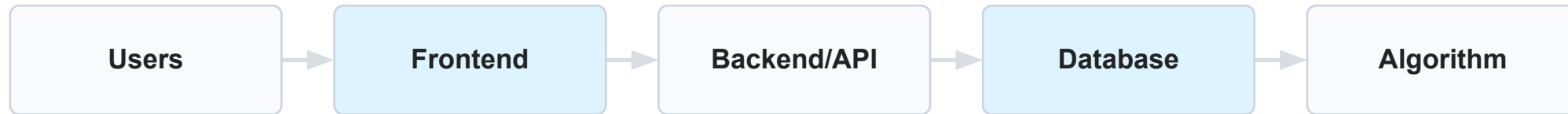
MVP test

Can a new user complete the core task in 2–3 minutes and get useful output?

Target: working end-to-end prototype by Week 4 of the ten remaining weeks, even if it only handles a few users or examples.

System architecture: start with a simple map

Every team should draw this and replace generic boxes with project-specific components.



**Add external services/APIs only after you verify access, limits, cost, and data quality.
Show how data flows through the system and where each team member contributes.
Mark risks: unknown API, data collection, privacy, model accuracy, deployment.**

Technology planning: known vs. new

Use what you know for most of the stack; learn only what the project truly needs.

Already know

Python, JavaScript, databases, Git/GitHub, basic web apps, APIs

Need to learn

Specific data source, external API, deployment, authentication, model/recommender, cloud service

Investigate first

Can we legally/technically access this data? Is there a free tier? What is the failure plan?

Decision rule

Choose boring, reliable tech unless the new tech is central to the project contribution.

This semester: backend, data, algorithms, testing, and integration matter more than frontend polish.

Sample 10-week MVP plan

Generic template: adapt this to your project.

Weeks 1–5

1. Lock MVP scope, architecture, team roles, risks
2. Database schema + backend API skeleton
3. Core data collection / ingestion path
4. End-to-end prototype with simple UI
5. Core algorithm / model / logic

Weeks 6–10

6. Integration + first real-user test
7. Improve based on feedback; add tests
8. Prepare code demo; stabilize deployment
9. Prepare presentation + writing evidence
10. Final demo script, evaluation, documentation

Four-person team: share the real work

Avoid one person doing “just frontend” while others own the technical core.

Student A: Backend/API owner

Also contributes to testing + writing.

Student B: Data/database owner

Also contributes to UI + presentation.

Student C: Algorithm/model owner

Also contributes to backend + evaluation.

Student D: Frontend/integration owner

Also contributes to data + demo script.

Each member should understand the whole system architecture.
Each member should have code they can explain during demos.
Rotate review: everyone reads another person’s pull request or module.

Plan backward from final deliverables

The project is not only code. You need evidence, explanation, and communication.

Final code demo

- Working core flow
- Clear demo script
- Real or realistic data
- Tests / error cases
- Explain limitations

Final presentation

- Problem and users
- Technical contribution
- Architecture
- Evaluation results
- Lessons learned

Final writing

- Motivation
- Related/competitive work
- Design decisions
- Technical details
- Ethical/social impact

Collect screenshots, design decisions, user feedback, and test results every week — do not wait until December.

Team Charter: use today's lab time

Build a living agreement for how your team will work.

Project summary for a non-technical audience
Goals and objectives
Roles and responsibilities
Communication guidelines

Decision-making guidelines
Conflict resolution strategies
Performance standards
Resource allocation

Reminder: each student signs their name through an individual GitHub commit.

[Writing assignment page](#)

Team Charter workshop prompts

Spend about 25–30 minutes drafting real answers as a team.

1. Collaboration

When do we meet? Which channel is urgent? How fast should teammates respond?

2. Work quality

What counts as “done”? How do we review code? How do we handle missing work?

3. Conflict

What if two members disagree? When do we involve mentor or instructor?

4. Resources

Data, APIs, hardware, cloud credits, test users.

5. AI usage

Which tools? What is allowed? How do we verify outputs?

6. Fairness

How will every student contribute to code, demo, writing, and presentation?

AI usage agreement

AI can accelerate work, but it cannot replace your understanding.

Allowed / encouraged

- Generate starter code
- Debug errors
- Explain unfamiliar APIs
- Draft tests
- Compare design options

Required

- Explain your code
- Verify AI output
- Test edge cases
- Cite/disclose when required
- Protect private data

Not acceptable

- Submit code you do not understand
- Paste AI-written writing as your own
- Hide major AI use
- Upload private data to AI tools

For writing and presentations: use AI only within assignment rules; your submitted explanations must be your own.

AI guidance resources

Three useful references for responsible course-level AI use.

GWU

Official guidance for AI tools in academic work. Students remain responsible for accuracy and attribution.

[GWU AI Guidelines](#)

MIT

Teaching guidance on defining AI use and keeping learning goals clear.

[MIT GenAI & Your Course](#)

Stanford

Course policy guidance: specify tools, conditions, rationale, and student responsibilities.

[Stanford AI Course Policy](#)

Team Charter prompt: list the AI tools your team may use and how you will check correctness.
Code: AI assistance is welcome. Writing: follow the course writing rules.

Job market: use Senior Design as evidence

Mock interviews help, but your project portfolio also matters.

Technical proof

- GitHub repo with README
- Architecture diagram
- Demo video / screenshots
- Tests and evaluation
- Clear commit history

Communication proof

- Resume bullet with impact
- 60-second project pitch
- Explain tradeoffs
- Explain your contribution
- Discuss failures honestly

Opportunity strategy

- Alumni and mentor advice
- Referrals when appropriate
- Research / open source
- Internships / part-time roles
- Graduate school options

Mock Interviews

Mini-round today; continue practicing outside lab.

1. Find a partner.
2. Choose one question each from neetcode.io or another approved practice site.
3. Spend 8 minutes learning the solution to your question.
4. Interview your partner for 15 minutes.
5. Switch roles later or continue after mentor scheduling.

neetcode.io

Interviewer: guide, ask about Big O, and watch communication. Interviewee: start with brute force and explain thinking.

Mock Interview Question Options

Choose different questions. Pick a level that fits the partner.

Easy

Valid Parentheses, Meeting Schedule, Last Stone Weight

Medium

Product of Array Except Self, Remove Node From Linked List, Unique Paths

Hard

Merge K Sorted Lists, Maximum Frequency Stack, IPO

Ask for time complexity and space complexity.

Practice communication, not just getting the final answer.

After the interview, give one strength and one improvement.

After 7:40: mentor meeting time

Use the last part of lab to meet remotely with your alumni mentor.

Before the call

- Find a study room
- Open your proposal
- Open your architecture plan
- Prepare 2–3 questions

During the call

- Explain MVP scope
- Ask about risks
- Confirm weekly meeting time
- Take notes

After the call

- Update GitHub/notes
- Assign follow-up tasks
- Add risks to sprint plan
- Share decisions with team

Goal for next meeting: mentor understands your MVP, architecture, team roles, and top risks.